arXiv:2508.04652v1 [cs.AI] 6 Aug 2025

LLM Collaboration With Multi-Agent Reinforcement Learning

Shuo Liu[1], **Zeyu Liang**[1], **Xueguang Lyu**[1], **Christopher Amato**[1*]

[1]Khoury College of Computer Sciences, Northeastern University
Boston, MA, 02115, USA

Abstract

A large amount of work has been done in Multi-Agent Systems (MAS) for modeling and solving problems with multiple interacting agents. However, most LLMs are pre-trained independently and not specifically optimized for coordination. Existing LLM fine-tuning frameworks rely on individual rewards, which require complex reward designs for each agent to encourage collaboration. To address these challenges, we model LLM collaboration as a cooperative Multi-Agent Reinforcement Learning (MARL) problem. We develop a multi-agent, multi-turn algorithm, Multi-Agent Group Relative Policy Optimization (MAGRPO), to solve it, building on current RL approaches for LLMs as well as MARL techniques. Our experiments on LLM writing and coding collaboration demonstrate that fine-tuning MAS with MAGRPO enables agents to generate high-quality responses efficiently through effective cooperation. Our approach opens the door to using other MARL methods for LLMs and highlights the associated challenges.

Introduction

Leveraging billions of parameters and extensive pre-training on large-scale datasets, state-of-the-art LLMs have demonstrated remarkable capabilities across diverse domains (Grattafiori et al. 2024; Achiam et al. 2023; Anil et al. 2025). To adapt to specific applications or align with human preferences, fine-tuning has emerged as a critical secondary training stage. Compared to supervised fine-tuning, Reinforcement Learning (RL) enables more generalizable learning for complex, multi-turn tasks through human-aligned reward design, making it an important technique for fine-tuning (Ouyang et al. 2022; Guo et al. 2025; Ziegler et al. 2020).

Likewise, Multi-Agent Systems (MAS) have been extensively studied over the past decades, with substantial progress in modeling and solving problems involving multiple agents (Littman 1994; Shoham and Leyton-Brown 2009; Stone and Veloso 2000). In particular, advances in cooperative MAS have demonstrated strong potential for enabling effective collaboration in distributed settings, such as games, robotics, and traffic control (Samvelyan et al. 2019; Vinyals et al. 2017; Berner et al. 2019; Amato et al. 2016; Wiering 2000). These developments motivate the application of MAS principles and techniques to LLM collaboration, where multiple LLMs working together can solve more complex tasks in a more robust and efficient manner.

There has been some recent work on coordinating multiple LLMs. Some approaches implement coordination at the inference stage, enabling agents to interact through debate, discussion, or verification (Du et al. 2023; Wu et al. 2023a; Lifshitz, McIlraith, and Du 2025). These methods operate at the prompt level, with fixed models that are not tuned toward coordination-centric objectives. The agents may have conflicting answers or spread incorrect information to other participants, limiting performance (Cemri et al. 2025; Estornell and Liu 2024). Moreover, the design of effective prompts remains difficult and unclear. Other approaches fine-tune agents independently with individual or role-conditioned rewards. However, they require carefully curated rewards for each individual or role (Slumbers et al. 2024; Liu et al. 2025; Subramaniam et al. 2025), and, as independent learning methods, lack convergence guarantees (Tan 1993).

In this paper, we model LLM collaboration as a cooperative MARL problem (Albrecht, Christianos, and Schäfer 2024) and formalize it as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP) (Oliehoek and Amato 2016). In LLM collaboration, multiple trainable LLMs generate responses synchronously based on their individual prompts. The external environment evolves according to the joint responses until the dialog ends. This general model allows a wide range of problems to be modeled and solved using versions of MARL algorithms. Following the efficient practice of Group Relative Policy Optimization (GRPO) (Guo et al. 2025), we propose Multi-Agent GRPO (MAGRPO) that trains LLMs in a multi-turn setting. MAGRPO leverages centralized group-relative advantages for joint optimization, while preserving decentralized execution for each agent. The resulting method builds off of state-of-the-art LLM approaches in GRPO and MARL approaches for centralized training and decentralized execution, such as MAPPO (Yu et al. 2022). Our experiments demonstrate that MAGRPO develops various LLM cooperation schemes, improving response efficiency with high quality.

Our contributions can be summarized as follows: (i) We model the LLM collaboration as a cooperative MARL problem, where multiple LLMs cooperate to generate joint responses; (ii) We implement the MAGRPO algorithm, which

* Corresponding author camato@ccs.neu.edu.

optimizes agent cooperation through aligned rewards while maintaining decentralized execution to maintain efficiency; (iii) Our experiments demonstrate that fine-tuning with MA-GRPO improves both response efficiency and quality in writing and coding collaboration; (iv) We provide a detailed analysis of the limitations of existing approaches and outline open challenges in applying MARL to LLM collaboration.

Related Work

Test-Time Multi-Agent Interaction Recent work employs multiple agents with specialized roles interacting through diverse pipelines at test-time to enhance response quality. In multi-agent debate, agents iteratively formulate positions by reviewing other agents' outputs, where the final decision or answer is determined by majority voting or a summarizer (Du et al. 2023; Chan et al. 2023; Liang et al. 2024). Role-based approaches allocate tasks across specialized agents (Wu et al. 2023a; Qian et al. 2024; Hong et al. 2024). For example, an agent may function as a verifier to assess the correctness of outputs (Skreta et al. 2023; Lifshitz, McIlraith, and Du 2025; Setlur et al. 2025), while another may act as a macro-planner to orchestrate workers' responses. However, these multi-agent frameworks rely on prompt-level interactions among agents, often leading to ineffective communication and computational inefficiency. Moreover, the design of effective prompts and role assignment remains unclear, as prompts usually fail to reliably guide agent behavior, enforce role adherence, or support coherent coordination across tasks. These limitations motivate us to fine-tune LLMs in MAS to improve their cooperation.

Multi-Agent Fine-Tuning Recent work has explored fine-tuning LLMs to improve their performance across diverse domains, e.g., arithmetic reasoning, navigation, and hidden-role games (Ma et al. 2025; Slumbers et al. 2024; Sarkar et al. 2025). These approaches typically employ individual rewards or rewards conditioned on specific roles (Park et al. 2025; Liu et al. 2025; Subramaniam et al. 2025). Such reward structures often require careful manual specification, and their underlying rationale is rarely well justified. The misaligned or conflicting incentives can hinder effective coordination. Moreover, these methods lack convergence guarantees, as each agent learns independently in a non-stationary environment where other agents are simultaneously updating their policies. In this paper, we focus on cooperative scenarios, where LLMs are jointly trained with interpretable, human-aligned rewards.

Cooperative MARL for LLM Collaboration

Since LLMs can be viewed as a special class of agents, we leverage advances in MAS to facilitate their collaboration. We model LLM collaboration as a cooperative MARL problem and outline its unique challenges. We formalize this problem as a Dec-POMDP, as shown in Figure 1.

LLM Collaboration

LLM collaboration refers to the problems where LLMs cooperatively solve a class of tasks in MAS. The tasks are specified in natural language and provided to the agents as prompts. Each LLM agent generates a response synchronously based on its individual instructions. The set of these responses jointly forms a solution to the task.

Most tasks cannot be resolved in one turn. Users, external models, or systems validate the solutions and provide additional requirements or suggestions for LLMs. These components also serve as part of the environment for LLM collaboration, whose states may change based on the agents' outputs. The updates are embedded into prompts for subsequent turns. This iterative process continues until the task is successfully completed or a predefined turn limit is reached.

As discussed by a number of companies (NVIDIA 2024; Anthropic 2024), a team of agents could be used to generate a complex codebase. The code would be difficult, costly, and time-consuming to generate with a single agent, but a group of LLMs could do so quickly and cheaply. None of these agents is self-interested, but they are trainable in a scheme such as the one discussed below. Using a joint reward allows agents to specialize as needed to complete the task without complex prompt or reward engineering.

Problem Formalization

We formalize collaboration among LLMs as a subclass of the cooperative MARL problem, considering LLM agents and the types of problems they are solving. This problem is a form of a Dec-POMDP (Oliehoek and Amato 2016), which allows cooperation through a joint reward while preserving scalable decentralized control. We show 2 instantiations of our framework in writing and coding tasks in the experiments section.

Mathematically, our LLM Dec-POMDP is defined by a tuple $\langle \mathcal{I}, \mathcal{S}, \{\mathcal{O}_i\}, \{\mathcal{A}_i\}, R, T, H \rangle$.

- $\mathcal{I} = \{1, \cdots, n\}$ denotes the set of n LLM agents, each instantiated with a pre-trained language model.
- $\mathcal{S}$ denotes the full global state space. At turn t, a full state $s_t = (s_t^{\text{acc}}, s_t^{\text{usr}})$ consists of parts that are accessible in the model and provided to the reward model $s_t^{\text{acc}} \in \mathcal{S}^{\text{acc}}$ (e.g., external models or systems), and the inaccessible user state $s_t^{\text{usr}} \in \mathcal{S}^{\text{usr}}$ which updates over time but isn't maintainable. In Dec-POMDP, the state is not directly observable by the agents (LLMs).
- $\mathcal{O}_i$ is the observation space for agent i with $\mathcal{O} = \times_i \mathcal{O}_i$ the joint observation space. A local observation $o_{i,t}$ consists of natural language instructions (i.e., prompts), providing a partial and noisy view of s_t.
- $\mathcal{A}_i$ is the action space for agent i with $\mathcal{A} = \times_i \mathcal{A}_i$ the joint action space. A local action $a_{i,t}$ is a response in natural language to the given prompt.
- $R : \mathcal{S}^{\text{acc}} \times \mathcal{A} \to \mathbb{R}$ is the joint reward function implemented via predefined rules or a pretrained reward model. At turn t, the joint rewards r_t are determined by the accessible part of current state s_t^{acc} and the agents' joint action $\mathbf{a}_t = \{a_{1,t}, \cdots, a_{n,t}\}$.
- $T : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ is the underlying stochastic state transition function. At turn t, the agents' joint actions $\mathbf{a}_t$ induce a shift to a new state $s_{t+1} \sim T(\cdot|s_t, \mathbf{a}_t)$, which reflects the updates in the user state and the states of external models and systems.

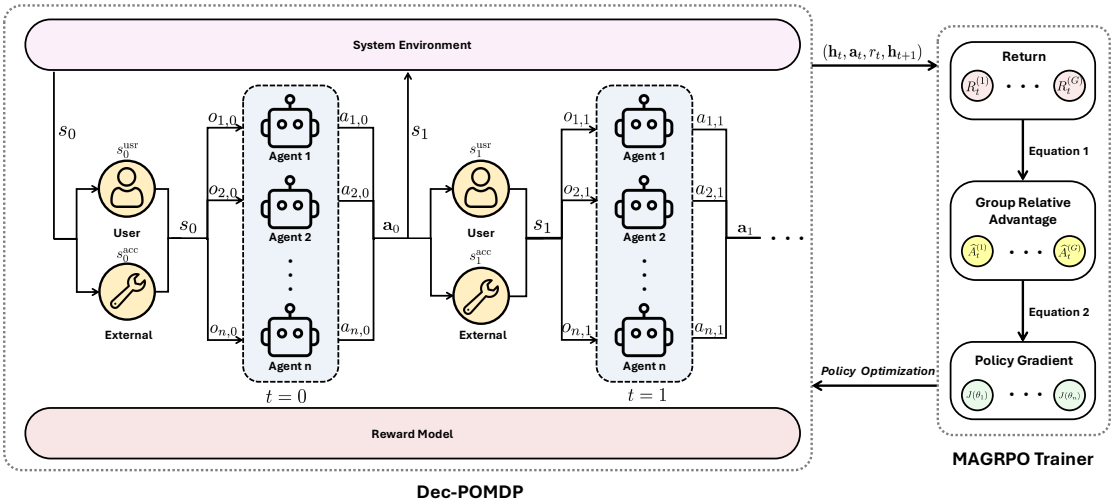


Figure 1: Illustration of Dec-POMDP and our MAGRPO algorithm.

- H is the episode horizon, i.e., the turn limit of the dialog.

In Dec-POMDP, since the states are not directly observed, each agent maintains its local observation-action history $\mathbf{h} = \{h_1, \cdots, h_n\}$ to infer information about state. A solution to a Dec-POMDP is a joint policy that maximizes the expected cumulative reward, $\boldsymbol{\pi}^* = \{\pi_1^*, \cdots, \pi_n^*\} = \arg\max_{\boldsymbol{\pi}} \mathbb{E}_{\boldsymbol{\pi}} \left[\sum_{t=0}^{H-1} R(s_t^{\text{acc}}, \mathbf{a}_t)\right]$. A joint polict is a set of local policies π_i, which conditions on the local observation-action history $h_{i,t} = \{o_{i,0}, a_{i,0}, \ldots, o_{i,t}\}$.

RL methods for Dec-POMDPs have become a popular topic (e.g., (Foerster et al. 2024; Lowe et al. 2020; Foerster et al. 2018; Rashid et al. 2018; Wang et al. 2021; Yu et al. 2022; Albrecht, Christianos, and Schäfer 2024)) with methods successful at scaling to large state, action and observation spaces. Many methods use Centralized Training for Decentralized Execution (CTDE), where they use some centralized information during training (e.g., a centralized value function estimate) but are still able to execute in a decentralized manner when training is complete.

Challenges in LLM Collaboration

LLM collaboration presents unique challenges compared to traditional MARL problems, where LLM agents receive and process tasks through natural language.

Representations in Natural Language Unlike traditional cooperative MARL agents, LLM agents operate over natural language, receiving instructions and generating responses as sequences of tokens. MARL approaches could model this problem at the token or prompt/response level. At the token level, the number of actions and observations is smaller, but the problem horizon can be very long. At the prompt/response level, the actions and observations space is much larger, but the horizon is much shorter. Moreover, token-level rewards are often uninformative, as both queries and responses must form coherent and semantically meaningful structures. As adopted in prior RL methods (Ouyang et al. 2022; Guo et al. 2025; Rafailov et al. 2024), we model each LLM agent's decision-making process as a direct mapping from input instructions to complete responses to enable efficient and stable training. Nevertheless, the best modeling and solution approaches remain an open question.

Training Paradigm As mentioned above, many MARL methods use centralized training for decentralized execution (CTDE). Unfortunately, standard CTDE methods use centralized value models in the form of centralized critics (Foerster et al. 2024; Lowe et al. 2020; Yu et al. 2022) or mixers in value decomposition methods (Rashid et al. 2018; Wang et al. 2021). Such architectures allow additional information and coordination during training but do not scale well to very large action and observation spaces (such as those in our problem). Conversely, Decentralized Training and Execution (DTE) methods (Amato 2025) train a set of models, one for each agent in a decentralized manner. DTE approaches are typically more scalable but don't use additional information during training (even when it is available). It is an open question which paradigm to use to maximize performance while maintaining scalability in the LLM collaboration problem. In this paper, we balance decentralized execution with centralized training using group-based Monte Carlo estimates. Experiments show the effectiveness of our approach on short-horizon tasks.

MAGRPO

We propose the Multi-Agent GRPO (MAGRPO) algorithm to jointly train LLM agents in MAS while maintaining decentralized execution.

Algorithm 1 shows the procedure of MAGRPO. Given a dataset $\mathcal{D}$ containing task information (e.g., the descriptions of coding problems), n LLMs are optimized, each with a policy parameterized by θ_i and guided by a reward model R. In each episode, a task is sampled from the given dataset $\mathcal{D}$, which is used to construct initial observations $\mathbf{o}_0 = \{o_{1,0}, \cdots, o_{n,0}\}$ and histories $\mathbf{h}_0 = \{h_{1,0}, \cdots, h_{n,0}\}$.

Algorithm 1: MAGRPO

Require: Dataset $\mathcal{D}$, n pretrained LLMs with policies $\{\pi_{\theta_1}, \cdots, \pi_{\theta_n}\}$, reward model R, generation group size G, learning rate α

for each episode **do**
  Sample a task $\sim \mathcal{D}$
  Initialize observations $o_{i,0}, \forall i \in \mathcal{I}$, according to the task, and $\mathbf{o}_0 = \{o_{1,0}, \cdots, o_{n,0}\}$
  $h^{\mathcal{G}}_{i,0} \leftarrow o_{i,0}, \forall i \in \mathcal{I}$, and $\mathbf{h}^{\mathcal{G}}_0 = \{h^{\mathcal{G}}_{1,0}, \cdots, h^{\mathcal{G}}_{n,0}\}$
  for turn $t = 0$ to $H-1$ **do**
    Generate a group of responses $a^{\mathcal{G}}_{i,t} \leftarrow \pi_{\theta_i}(\cdot|h^{\mathcal{G}}_{i,t})$, $\forall i \in \mathcal{I}$, where $h^{\mathcal{G}}_{i,t} = \{h^{(1)}_{i,t}, \cdots, h^{(G)}_{i,t}\}$, $a^{\mathcal{G}}_{i,t} = \{a^{(1)}_{i,t}, \cdots, a^{(G)}_{i,t}\}$, and $\mathbf{a}^{\mathcal{G}}_0 = \{a^{\mathcal{G}}_{1,t}, \cdots, a^{\mathcal{G}}_{n,t}\}$
    Obtain a joint reward $r^{\mathcal{G}}_t$ from system
    Receive new observations $o^{\mathcal{G}}_{i,t+1}$, and update history $h^{\mathcal{G}}_{i,t+1} \leftarrow \{h^{\mathcal{G}}_{i,t}, a^{\mathcal{G}}_{i,t}, o^{\mathcal{G}}_{i,t+1}\}, \forall i \in \mathcal{I}$
  end for
  for turn $t = H-1$ to 0 **do**
    Calculate return $R^{(g)}_t \leftarrow \sum_{\tau=t}^{H-1} r^{(g)}_\tau, \forall g \in \mathcal{G}$
    Estimate $\widehat{A}^{(g)}_t, \forall g \in \mathcal{G}$ according to Equation 1
    Calculate $J(\theta_i), \forall i \in \mathcal{I}$ according to Equation 2
    $\theta_i \leftarrow \theta_i + \alpha \nabla_{\theta_i} J(\theta_i), \forall i \in \mathcal{I}$
  end for
end for
return $\boldsymbol{\pi_\theta} = \{\pi_{\theta_1}, \cdots, \pi_{\theta_n}\}$

Taking inspiration from the single-agent GRPO algorithm (Guo et al. 2025), at each turn t, each agent takes action by generating a group of responses $a^{\mathcal{G}}_{i,t} = \{a^{(1)}_{i,t}, \cdots, a^{(G)}_{i,t}\}$ following its policy $\pi_i(\cdot|h^{\mathcal{G}}_{i,t})$ based on its observation-action history $h^{\mathcal{G}}_{i,t} = \{h^{(1)}_{i,t}, \cdots, h^{(G)}_{i,t}\}$. The actions of individual agents are aggregated to form a group of joint actions $\mathbf{a}^{\mathcal{G}}_t = \{a^{\mathcal{G}}_{0,t}, \cdots, a^{\mathcal{G}}_{n,t}\}$. The agents receive a group of joint rewards $r^{\mathcal{G}}_t$ for their responses $\mathbf{a}^{\mathcal{G}}_t$), which also conditions on the accessible part of the state $R(\cdot|s^{\text{acc},\mathcal{G}}_t, \mathbf{a}^{\mathcal{G}}_t)$. The joint actions triggers the transition $T(\cdot|s^{\mathcal{G}}_t, \mathbf{a}^{\mathcal{G}}_t)$, where agents receive new observations $o^{\mathcal{G}}_{i,t+1} = \{o^{(1)}_{i,t}, \cdots, o^{(G)}_{i,t}\}$ and use them to construct histories $h^{\mathcal{G}}_{i,t+1} = \{h^{\mathcal{G}}_{i,t}, a^{\mathcal{G}}_{i,t}, o^{\mathcal{G}}_{i,t+1}\}$. This process continues until terminated at turn H.

We employ stochastic gradient descent to train agents at the end of each episode. Without explicit value models, estimating history-action values from a single rollout incurs high variance. To stabilize training, we estimate the expected return of the current state by averaging over a group of Monte Carlo samples $\{R^{(1)}_t, \cdots, R^{G}_t\}$. As a result, we are able to generate a centralized estimate (which is common in MARL) without a large value model. For each turn t, the advantage of each joint action in the group is calculated as,

$$\widehat{A}^{(g)}_t = \frac{R^{(g)}_t - \frac{1}{G}\sum_{g=1}^{G} R^{(g)}_t}{\sigma(R^{(g)}_t)}, \tag{1}$$

where $\sigma(R^{\mathcal{G}}_t)$ represents the standard deviation of a group of expected returns, and $R^{(g)}_t = \sum_{\tau=t}^{H-1} r^{(g)}_\tau$.

Inspired by GRPO (Guo et al. 2025) and MAPPO (Yu et al. 2022), the centralized advantage values can be used to update policy π_i (parameterized by θ_i) for each agent i,

$$J(\theta_i) = \mathbb{E}_{\mathbf{o}_0 \sim \mathcal{D}, \mathbf{h}^{\mathcal{G}} \sim \boldsymbol{\pi}_{\boldsymbol{\theta},\text{old}}} \left[\frac{1}{|B|} \frac{1}{|\mathcal{G}|} \sum_{h^{\mathcal{G}}_i \in B} \sum_{g \in \mathcal{G}} \min\left(\rho^{(g)}_{i,t} \widehat{A}^{(g)}_t, \varepsilon\text{-clip}(\rho^{(g)}_{i,t}) \widehat{A}^{(g)}_t \right) \right], \tag{2}$$

where $\rho^{(g)}_{i,t} = \frac{\pi_{\theta_i}(a^{(g)}_{i,t}|h^{(g)}_{i,t})}{\pi_{\theta_i,\text{old}}(a^{(g)}_{i,t}|h^{(g)}_{i,t})}$ denotes the importance sampling ratio between the updated and previous policies.

Experiments

We evaluate MAGRPO on LLM writing and coding collaboration. Datasets, reward specifications, and additional results are provided in the Appendix.

Writing Collaboration

We explore LLM collaboration for article writing using MAGRPO across 2 classic tasks: summarization and expansion.

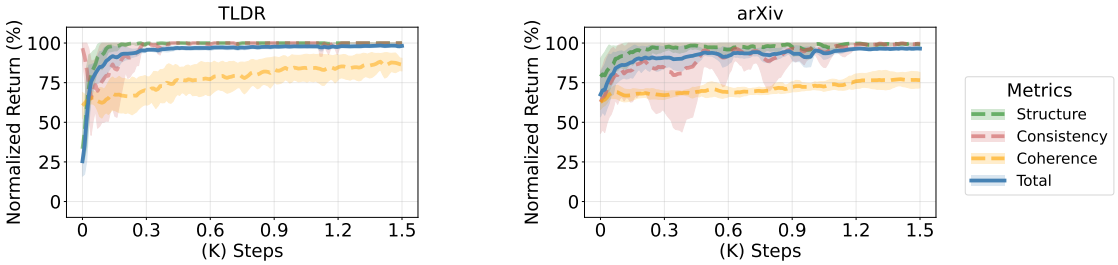
TLDR Summarization When reading a long article, readers often seek to quickly grasp its core idea. If the topic is of interest, they may wish to delve deeper into specific details while still avoiding a complete reading through the full document. This calls for a summarization system to generate summaries at varying levels of detail. We frame this task using TLDR summarization as an illustrative example.

The TLDR dataset comprises unabridged Reddit posts in the `prompt` and concise summaries appended by the author in the `completion`. In our experiment, 2 *Qwen3-1.7B* agents independently summarize the `prompt` without using `completion`. The first agent functions as a core-idea (TLDR) generator, producing a concise paragraph, while the second agent serves as a detailed summarizer, providing more comprehensive information.

To quantify the summarization quality, we employ a relatively simple combination of 3 metrics. Structure measures the lengths and the length ratio of the two summaries, to ensure the TLDR is concise and the detailed summary is sufficiently long. Style consistency is assessed using the normalized Jaccard similarity coefficient, calculated as the ratio of the intersection size to the union size of unique words (or n-grams) between responses. A high style consistency reward typically indicates that the summarizers adopt similar stylistic patterns while avoiding identical wording. Logical coherence is quantified by counting the occurrences of transition words. Positive reward is given for using transition words, but the reward decreases logarithmically as more are used. These metrics are simple approximations of what more complex reward models may evaluate. Other (simpler or more complex) metrics or reward models could also be used. The total reward combines these metrics through a weighted summation. More details regarding our reward model and hyperparameters are provided in the Appendix.

Method	Dataset	Efficiency		Article Quality (%)			Return (%)
		Speed	Response Time	Structure	Consistency	Coherence	
Single Model	TLDR	64.1	6.6	43.8	97.6	52.8	36.7
	arXiv	65.4	6.5	51.2	87.2	**71.1**	44.9
Parallel Generation	TLDR	185.6	**2.1**	25.9	98.3	56.5	23.2
	arXiv	190.6	**2.1**	71.5	64.2	61.5	59.6
Sequential Generation	TLDR	98.7	4.3	33.5	98.5	64.5	21.7
	arXiv	85.8	4.3	92.4	**97.8**	64.3	87.7
One-Round Discussion	TLDR	100.4	4.3	35.9	**98.8**	60.8	22.3
	arXiv	95.4	4.3	84.6	71.8	66.0	76.6
MAGRPO (Ours)	TLDR	**202.3**	**2.1**	**98.7**	97.1	**78.5**	**94.5**
	arXiv	**193.8**	**2.1**	**97.9**	96.2	69.7	**93.1**

Table 1: Performance of MAGRPO against baselines on TLDR and arXiv. Speed (tokens/s) and response time (s) are measured on GeForce RTX 5090s. Results are normalized within the return scale. **Bolds** indicate the best performance on each dataset.



(a) TLDR summarization

(b) arXiv abstract expansion

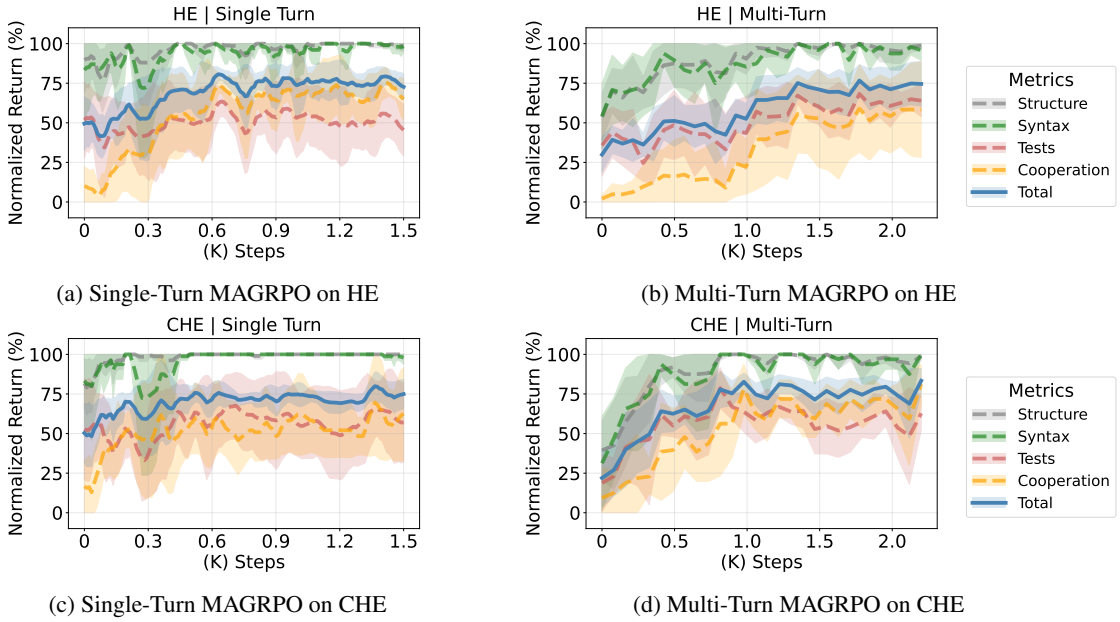
Figure 2: Normalized returns on writing collaboration: (a) structural wellness (dashed green); (b) style consistency (dashed red); (c) coherence (dashed orange); (d) total rewards (solid blue). All returns are normalized within the return scale.

arXiv Expansion Writing a long article typically requires contributions from multiple writers, each responsible for different sections. As a simple scenario, 2 agents can collaborate to generate introductions from the `abstract` of arXiv papers. The first agent outlines the research background and motivation, while the other presents the proposed methods and their experiments. The combined paragraphs should be coherent and consistent in style. Similar to the reward model in TLDR summarization, we employ the same evaluation metrics as proxies, with threshold hyperparameters specifically adjusted for this task.

Baselines We adopt a single-agent model and 3 representative multi-agent methods as our baselines. To minimize the influence of prompts on our comparison, we keep the task description fixed and only add minimal coordination instructions. Specifically, for the single-agent baseline, we prompt with the article to be manipulated, the agent's role (summarizer or expanding writer), and specific user instructions (e.g., format requirements). Naive concatenation builds on it by dividing the task into subtasks, assigning each agent a specific portion to complete in parallel without explicit communication. The sequential pipeline introduces one-way communication, allowing one agent to respond based on both the task description and the other agent's output. The one-round discussion baseline enables bidirectional communication: agents first receive the same prompts as in naive concatenation, then the prompts are augmented with the other's first-turn response in the second turn. All baseline methods operate without fine-tuning and depend solely on prompt-level interactions. Detailed prompts for each baseline are provided in the Appendix.

Results In this experiment, we apply MAGRPO to optimize the dual *Qwen3-1.7B* system in one turn. Figure 2a and Figure 2b show the evaluation results on TLDR and arXiv over 5 runs. The upward trend on all metric curves indicates that 2 agents gradually cooperate to generate coherent and consistent content with a well-organized structure. In the TLDR summarization, while the structure and logical coherence monotonically increase throughout training, the style consistency curves exhibit a decrease in the first 100 steps. This occurs as agents temporally diverge in styles to optimize other cooperative objectives, but their styles would be gradually realigned and stabilized with sufficient training.

As shown in Table 1, MAGRPO is 3 times faster compared to the single *Qwen3-4B* model, which has a comparable number of parameters to our dual *Qwen3-1.7B* system. Despite receiving detailed instructions, *Qwen3-4B* fails to produce well-structured responses. A similar issue appears in TLDR summarization but not in arXiv expansion under multi-agent settings. This is because the outputs of homogeneous agents are naturally similar in length, which fortuitously aligns with the preference of the reward model.



(a) Single-Turn MAGRPO on HE

(b) Multi-Turn MAGRPO on HE

(c) Single-Turn MAGRPO on CHE

(d) Multi-Turn MAGRPO on CHE

Figure 3: Normalized returns on coding collaboration: (a) structural wellness (dashed grey); (b) syntax correctness (dashed green); (c) Test score (dashed red); (d) cooperation rewards (dashed yellow); (e) total return (solid blue).

Among the multi-agent baselines, parallel generation is the only one that achieves a comparable speed to ours, but it fails to generate well-structured and coherent texts due to the lack of cooperation. Sequential generation and discussion-based approaches occasionally enhance coordination through specific prompts. However, they still underperform ours in efficiency and coherence, resulting in lower total return. The limited effectiveness of prompt-instructed coordination constrains their scalability to more complex scenarios involving large numbers of agents or extended multi-turn interactions (Estornell and Liu 2024).

Coding Collaboration

In large-scale software development, numerous developers collaborate to implement complex systems. Employing LLMs as developers is a promising direction, but coordinating them remains challenging due to diverse cooperation schemes and complex failure modes. In our experiments, we simplify this task by using 2 *Qwen2.5-Coder-3B* agents to generate Python functions collaboratively. A helper agent produces auxiliary functions to support a main function generator, without any direct communication. The outputs from both agents, along with required libraries, are aggregated into complete code snippets.

HumanEval We evaluate MAGRPO on the HumanEval (HE) dataset, which comprises 164 handwritten programming problems, each containing a natural language description (`prompt`), a function signature (`entry_point`), and a set of unit tests (`test`). To guide learning, we design a level-based reward model that prioritizes fundamental aspects of code generation. Structural integrity verifies the presence and correctness of both main and auxiliary function definitions; syntactic correctness ensures compliance with Python syntax; test pass rate assesses functional correctness based on the proportion of successfully passed unit tests; and a cooperation quality bonus is granted when the main function properly invokes and utilizes the auxiliary function. Rewards are accumulated only when all requirements at each preceding level are satisfied.

CoopHumanEval Most entries in HumanEval (HE) are not designed for coding collaboration; certain atomic operations (e.g., `strlen(string)`) can hardly be decomposed in a way that facilitates meaningful cooperation. These noisy instances introduce instability into training or bias it toward invalid cooperation schemes, such as merely wrapping the auxiliary function. Thus, we construct a cooperation-oriented code generation dataset, CoopHumanEval (CHE), which comprises both original HE problems with cooperative potential (e.g., `prime_fib(n)`) and additional handwritten tasks (e.g., `compare_areas(shapes)`). The problems in CHE are readily decomposable, enabling agents to explore more effective cooperation.

Baselines We adopt a single *Qwen2.5-Coder-7B* model and 3 multi-agent methods as our baselines. In the single-agent baseline, the model generates a function based on the `prompt` and the specified `entry_point`. Multi-agent methods use two *Qwen2.5-Coder-3B* agents: one generates an unconstrained auxiliary function "aux", and the other produces the main function. Based on their roles, agents may generate independently (naive concatenation), sequentially (main agent receives auxiliary output), or with one round of revision using each other's initial responses. All baselines operate without additional training of the agents.

Method	Dataset	Efficiency		Code Quality (%)				Return (%)
		Speed	Response Time	Structure	Syntax	Tests	Cooperation	
Single *Qwen2.5-Coder-7B*	HE	73.1	1.6	**100.0**	**100.0**	64.8	–	–
	CHE	65.5	1.4	**100.0**	**100.0**	63.4	–	–
Naive Concatenation	HE	**194.9**	**1.1**	96.1	90.6	42.5	22.7	53.9
	CHE	189.4	**1.1**	97.5	95.0	40.1	24.0	54.3
Sequential Pipeline	HE	99.6	2.2	98.4	96.5	56.4	35.1	63.1
	CHE	97.4	2.0	97.5	96.3	55.2	35.2	62.5
One-Round Discussion	HE	82.5	2.8	98.1	94.8	41.2	30.2	57.5
	CHE	78.3	2.8	97.5	96.3	41.9	34.8	59.5
Single-Turn MAGRPO (Ours)	HE	190.0	1.5	100.0	97.8	61.6	83.4	83.7
	CHE	**192.4**	1.5	**98.8**	97.5	71.2	83.7	86.0
Multi-Turn MAGRPO (Ours)	HE	95.2	2.8	99.9	97.3	67.9	**84.9**	**85.8**
	CHE	97.3	2.5	99.8	97.9	**74.6**	**86.2**	**88.1**

Table 2: Performance comparison of MAGRPO against baselines on HE and CHE. Speed (tokens/s) and response time (s) are recorded on GeForce RTX 5090s. Results are normalized within the return scale and averaged over 5 runs; rewards are level-based. **Bolds** indicate the best performance of each metric on each dataset.

Results We optimize the interaction between 2 agents using single-turn and multi-turn MAGRPO. To reduce prompt-induced variance, the same prompt of naive concatenation is adopted in the first turn. In the multi-turn setting, the problem description and the agents' initial responses are provided to a *Claude-Sonnect-4* model, which generates feedback for each agent. The feedback could include functionality analysis, detected errors, and revision suggestions. Figure 3a and Figure 3b show the normalized return of MAGRPO on HE over 5 runs. Single-turn MAGRPO training improves the syntactical correctness and develops valid cooperation, while the test pass rate does not show much progress. As for the multi-turning training, agents are initially overwhelmed by the external model's feedback, resulting in even lower initial returns. They gradually adopt the suggestions and improve their returns. However, the test pass rate shows no significant improvement over the single-agent model, due to noisy entries in the dataset and hence unreliable feedback. This reflects the complexity and delicacy of coder coordination, where the main agent must accurately infer the functionality of auxiliary modules and trust their correctness without direct communication.

The performance of single-turn and multi-turn MAGRPO on the CHE dataset is shown in Figures 3c and 3d. Results show that MAGRPO achieves higher overall rewards and lower variance when trained on CHE over HE. In the multi-turn setting, although agents initially struggle to interpret the feedback, similarly to training on HE, the normalized returns gradually improve and eventually surpass those in the single-turn training. This demonstrates that, when trained on a dataset with well-defined cooperative structures and supported by reliable suggestions, agents can learn to incorporate such feedback to improve the quality of their responses.

Table 2 presents a performance comparison between MAGRPO and baselines on both HE and CHE. Compared to a single model, the naive concatenation method has lower test pass rates, as the main agent may generate codes based on incorrect assumptions about auxiliary functionality. In the sequential pipeline method, the main agent can provide a backup for the auxiliary function when it identifies potential vulnerabilities, thereby improving the test pass rate. However, this comes at the cost of slower inference speed. Although the one-round discussion method involves more communication between agents, its effectiveness remains limited due to potential misaligned cross-adaptation. MAGRPO outperforms all baselines by facilitating effective cooperation and leveraging feedback from the external model. Additional results, including pass@k, are present to validate these findings in the Appendix.

Cooperation Schemes MAGRPO identifies diverse cooperation schemes. In some cases, the auxiliary function handles the core logic, while the main agent adds backup logic or decorations to improve the overall solution. Alternatively, the main agent may act as a coordinator, decomposing the problem and assigning subtasks to the auxiliary agent. The auxiliary function may serve as a strategy filter, guiding the main agent to generate code for specific cases. While coordinator and strategy-filter schemes can improve inference efficiency, they are more prone to syntax and logical errors. With limited cooperation-oriented training data, the main agent typically resorts to more conservative roles, i.e., fallback or decoration. These cooperation schemes emerge during training under a relatively simple joint reward. More refined design patterns can be found when training agents to develop large-scale coding projects. Detailed analyses of cooperation schemes are provided in the Appendix.

Conclusion

In this paper, we model LLM collaboration as a cooperative MARL problem and formalize it by Dec-POMDP. We propose the MAGRPO algorithm to optimize agent cooperation through aligned rewards. Our experiments in coding and writing collaboration show that MAGRPO enables agents to generate high-quality responses via effective collaboration. Our work encourages future exploration of MARL-based methods for scalable and robust LLM collaboration.

References

Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Albrecht, S. V.; Christianos, F.; and Schäfer, L. 2024. *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press.

Amato, C. 2025. An Initial Introduction to Cooperative Multi-Agent Reinforcement Learning. arXiv:2405.06161.

Amato, C.; Konidaris, G.; Anders, A.; Cruz, G.; How, J. P.; and Kaelbling, L. P. 2016. Policy search for multi-robot coordination under uncertainty. *The International Journal of Robotics Research*, 35(14): 1760–1778.

Anil, R.; Borgeaud, S.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; Millican, K.; Silver, D.; et al. 2025. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805.

Anthropic. 2023. Collective Constitutional AI: Aligning a Language Model with Public Input.

Anthropic. 2024. How We Built a Multi-Agent Research System.

Berner, C.; Brockman, G.; Chan, B.; Cheung, V.; Debiak, P.; Dennison, C.; Farhi, D.; Fischer, Q.; Hashme, S.; Hesse, C.; et al. 2019. Dota 2 with Large Scale Deep Reinforcement Learning. arXiv:1912.06680.

Cemri, M.; Pan, M. Z.; Yang, S.; Agrawal, L. A.; Chopra, B.; Tiwari, R.; Keutzer, K.; Parameswaran, A.; Klein, D.; Ramchandran, K.; Zaharia, M.; Gonzalez, J. E.; and Stoica, I. 2025. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657.

Chan, C.-M.; Chen, W.; Su, Y.; Yu, J.; Xue, W.; Zhang, S.; Fu, J.; and Liu, Z. 2023. ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate. arXiv:2308.07201.

Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Oliveira Pinto, H. P.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.

Du, Y.; Li, S.; Torralba, A.; Tenenbaum, J. B.; and Mordatch, I. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. arXiv:2305.14325.

Estornell, A.; and Liu, Y. 2024. Multi-LLM Debate: Framework, Principals, and Interventions. In *Neural Information Processing Systems (NeurIPS)*.

Estornell, A.; Ton, J.-F.; Taufiq, M. F.; and Li, H. 2025. How to Train a Leader: Hierarchical Reasoning in Multi-Agent LLMs. arXiv:2507.08960.

Foerster, J.; Farquhar, G.; Afouras, T.; Nardelli, N.; and Whiteson, S. 2024. Counterfactual Multi-Agent Policy Gradients. arXiv:1705.08926.

Foerster, J. N.; Chen, R. Y.; Al-Shedivat, M.; Whiteson, S.; Abbeel, P.; and Mordatch, I. 2018. Learning with Opponent-Learning Awareness. arXiv:1709.04326.

Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Vaughan, A.; et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783.

Guo, D.; Yang, D.; Zhang, H.; Song, J.; Zhang, R.; Xu, R.; Zhu, Q.; Ma, S.; Wang, P.; Bi, X.; et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.

Hong, S.; Zhuge, M.; Chen, J.; Zheng, X.; Cheng, Y.; Zhang, C.; Wang, J.; Wang, Z.; Yau, S. K. S.; Lin, Z.; Zhou, L.; Ran, C.; Xiao, L.; Wu, C.; and Schmidhuber, J. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv:2308.00352.

Liang, T.; He, Z.; Jiao, W.; Wang, X.; Wang, Y.; Wang, R.; Yang, Y.; Shi, S.; and Tu, Z. 2024. Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate. arXiv:2305.19118.

Lifshitz, S.; McIlraith, S. A.; and Du, Y. 2025. Multi-Agent Verification: Scaling Test-Time Compute with Multiple Verifiers. arXiv:2502.20379.

Littman, M. L. 1994. Markov games as a framework for multi-agent reinforcement learning. In Cohen, W. W.; and Hirsh, H., eds., *Machine Learning Proceedings 1994*, 157–163. San Francisco (CA): Morgan Kaufmann. ISBN 978-1-55860-335-6.

Liu, B.; Guertler, L.; Yu, S.; Liu, Z.; Qi, P.; Balcells, D.; Liu, M.; Tan, C.; Shi, W.; Lin, M.; Lee, W. S.; and Jaques, N. 2025. SPIRAL: Self-Play on Zero-Sum Games Incentivizes Reasoning via Multi-Agent Multi-Turn Reinforcement Learning. arXiv:2506.24119.

Lowe, R.; Wu, Y.; Tamar, A.; Harb, J.; Abbeel, P.; and Mordatch, I. 2020. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. arXiv:1706.02275.

Ma, H.; Hu, T.; Pu, Z.; Liu, B.; Ai, X.; Liang, Y.; and Chen, M. 2025. Coevolving with the Other You: Fine-Tuning LLM with Sequential Cooperative Multi-Agent Reinforcement Learning. arXiv:2410.06101.

NVIDIA. 2024. Introduction to LLM Agents.

Oliehoek, F. A.; and Amato, C. 2016. *A Concise Introduction to Decentralized POMDPs*. Springer.

Oliehoek, F. A.; Spaan, M. T. J.; and Vlassis, N. 2008. Optimal and Approximate Q-value Functions for Decentralized POMDPs. *Journal of Artificial Intelligence Research*, 32: 289–353.

Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35: 27730–27744.

Park, C.; Han, S.; Guo, X.; Ozdaglar, A.; Zhang, K.; and Kim, J.-K. 2025. MAPoRL: Multi-Agent Post-Co-Training for Collaborative Large Language Models with Reinforcement Learning. arXiv:2502.18439.

Qian, C.; Liu, W.; Liu, H.; Chen, N.; Dang, Y.; Li, J.; Yang, C.; Chen, W.; Su, Y.; Cong, X.; Xu, J.; Li, D.; Liu, Z.; and Sun, M. 2024. ChatDev: Communicative Agents for Software Development. arXiv:2307.07924.

Rafailov, R.; Sharma, A.; Mitchell, E.; Ermon, S.; Manning, C. D.; and Finn, C. 2024. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290.

Rashid, T.; Samvelyan, M.; de Witt, C. S.; Farquhar, G.; Foerster, J.; and Whiteson, S. 2018. QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning. arXiv:1803.11485.

Samvelyan, M.; Rashid, T.; de Witt, C. S.; Farquhar, G.; Nardelli, N.; Rudner, T. G. J.; Hung, C.-M.; Torr, P. H. S.; Foerster, J.; and Whiteson, S. 2019. The StarCraft Multi-Agent Challenge. arXiv:1902.04043.

Sarkar, B.; Xia, W.; Liu, C. K.; and Sadigh, D. 2025. Training Language Models for Social Deduction with Multi-Agent Reinforcement Learning. arXiv:2502.06060.

Setlur, A.; Rajaraman, N.; Levine, S.; and Kumar, A. 2025. Scaling Test-Time Compute Without Verification or RL is Suboptimal. arXiv:2502.12118.

Shoham, Y.; and Leyton-Brown, K. 2009. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge, UK: Cambridge University Press. ISBN 9780521899437.

Skreta, M.; Yoshikawa, N.; Arellano-Rubach, S.; Ji, Z.; Kristensen, L. B.; Darvish, K.; Aspuru-Guzik, A.; Shkurti, F.; and Garg, A. 2023. Errors are Useful Prompts: Instruction Guided Task Programming with Verifier-Assisted Iterative Prompting. arXiv:2303.14100.

Slumbers, O.; Mguni, D. H.; Shao, K.; and Wang, J. 2024. Leveraging Large Language Models for Optimised Coordination in Textual Multi-Agent Reinforcement Learning.

Stone, P.; and Veloso, M. 2000. Multiagent Systems: A survey from a machine learning perspective. *Autonomous Robots*, 8(3): 345–383.

Subramaniam, V.; Du, Y.; Tenenbaum, J. B.; Torralba, A.; Li, S.; and Mordatch, I. 2025. Multiagent Finetuning: Self Improvement with Diverse Reasoning Chains. arXiv:2501.05707.

Tan, M. 1993. Multi-Agent Reinforcement Learning: Independent versus Cooperative Agents. In *Proceedings of the Tenth International Conference on Machine Learning*, 330–337. San Francisco, CA, USA: Morgan Kaufmann. ISBN 1-55860-307-7.

Uesato, J.; Kushman, N.; Kumar, R.; Song, F.; Siegel, N.; Wang, L.; Creswell, A.; Irving, G.; and Higgins, I. 2022. Solving math word problems with process- and outcome-based feedback. arXiv:2211.14275.

Vinyals, O.; Ewalds, T.; Bartunov, S.; Georgiev, P.; Vezhnevets, A. S.; Yeo, M.; Makhzani, A.; Küttler, H.; Agapiou, J.; Schrittwieser, J.; Quan, J.; Gaffney, S.; Petersen, S.; Simonyan, K.; Schaul, T.; van Hasselt, H.; Silver, D.; Lillicrap, T.; Calderone, K.; Keet, P.; Brunasso, A.; Lawrence, D.; Ekermo, A.; Repp, J.; and Tsing, R. 2017. StarCraft II: A New Challenge for Reinforcement Learning. arXiv:1708.04782.

Wang, B.; Zi, Y.; Sun, Y.; Zhao, Y.; and Qin, B. 2024. RKLD: Reverse KL-Divergence-based Knowledge Distillation for Unlearning Personal Information in Large Language Models. arXiv:2406.01983.

Wang, J.; Ren, Z.; Liu, T.; Yu, Y.; and Zhang, C. 2021. QPLEX: Duplex Dueling Multi-Agent Q-Learning. arXiv:2008.01062.

Wiering, M. A. 2000. Multi-agent reinforcement learning for traffic light control. In *Proceedings of the Seventeenth International Conference on Machine Learning (ICML'2000)*, 1151–1158. Stanford, CA, USA: Morgan Kaufmann.

Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; Awadallah, A. H.; White, R. W.; Burger, D.; and Wang, C. 2023a. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.

Wu, Z.; Hu, Y.; Shi, W.; Dziri, N.; Suhr, A.; Ammanabrolu, P.; Smith, N. A.; Ostendorf, M.; and Hajishirzi, H. 2023b. Fine-Grained Human Feedback Gives Better Rewards for Language Model Training. arXiv:2306.01693.

Yu, C.; Velu, A.; Vinitsky, E.; Gao, J.; Wang, Y.; Bayen, A.; and Wu, Y. 2022. The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games. arXiv:2103.01955.

Zhao, P.; Sun, F.; Shen, X.; Yu, P.; Kong, Z.; Wang, Y.; and Lin, X. 2024. Pruning Foundation Models for High Accuracy without Retraining. arXiv:2410.15567.

Ziegler, D. M.; Stiennon, N.; Wu, J.; Brown, T. B.; Radford, A.; Amodei, D.; Christiano, P.; and Irving, G. 2020. Fine-Tuning Language Models from Human Preferences. arXiv:1909.08593.

Formalization of Multi-Agent Interaction

Many studies adopt Partially Observable Stochastic Games (POSG) to model the LLM interaction in MAS (Slumbers et al. 2024; Park et al. 2025; Liu et al. 2025; Sarkar et al. 2025). In this section, we show that Dec-POMDP offers special merits compared to POSG in the solution concept in the cooperative settings, thus more suited to model LLM collaboration.

Dec-POMDP

A Dec-POMDP is defined by $\langle \mathcal{I}, \mathcal{S}, \{\mathcal{O}_i\}, \{\mathcal{A}_i\}, R, T, H \rangle$. At each step t, since an agent cannot directly observe the state s_t, it usually maintains local observation-action history $h_{i,t} = (o_{i,0}, a_{i,0}, \ldots, o_{i,t})$ to infer a belief over the underlying state. Decisions are made according to a local policy $\pi_i : \mathcal{H}_{i,t} \to \Delta(\mathcal{A}_i)$, which maps histories to probability distributions over actions. The set of all local policies forms the joint policy $\boldsymbol{\pi} = \{\pi_1, \ldots, \pi_n\}$. In cooperative settings, the objective is to maximize shared cumulative rewards. As proved in (Oliehoek, Spaan, and Vlassis 2008), there is always an optimal joint policy in a Dec-POMDP,

$$\boldsymbol{\pi}^* = \operatorname*{argmax}_{\boldsymbol{\pi} \in \boldsymbol{\Pi}} \mathbb{E}_{\boldsymbol{\pi}} \left[\sum_{t=0}^{H-1} R(s_t, a_t) \right]. \tag{3}$$

POSG

A Partially Observable Stochastic Game (POSG), so-called Partially Observable Markov Game (POMG), does not assume cooperative behavior among agents. It can be either a cooperative, competitive, or mixed game. A POSG is defined as $\langle \mathcal{I}, \mathcal{S}, \{\mathcal{A}_i\}, T, \{\mathcal{O}_i\}, O, \{R_i\}, H \rangle$, where each agent has its own reward function $R_i : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. In POSG, each agent seeks to maximize its individual return under the fixed policies of all others $\boldsymbol{\pi}_{-i}$. The optimal policy $\pi_i^{\circledast}$ for each agent $i \in \mathcal{I}$ is,

$$\pi_i^{\circledast} = \operatorname*{argmax}_{\pi_i \in \Pi_i} \mathbb{E}_{\pi_i, \boldsymbol{\pi}_{-i}} \left[\sum_{t=0}^{H-1} R_i(s_t, a_t) \right], \tag{4}$$

The solutions for POSG are Nash Equilibria (NE), where no agents can unilaterally improve their returns by deviating from their policies. Formally, for all $i \in \mathcal{I}$ and any alternative policy $\pi_i \in \Pi_i$, NE satisfy

$$\mathbb{E} \left[\sum_{t=0}^{H-1} R_i(s_t, a_t) \mid \pi_i^{\circledast}, \boldsymbol{\pi}_{-i}^{\circledast} \right] \geq \mathbb{E} \left[\sum_{t=0}^{H-1} R_i(s_t, a_t) \mid \pi_i, \boldsymbol{\pi}_{-i}^{\circledast} \right]. \tag{5}$$

Like Dec-POMDP, the decision-making in POSG is still concurrent (as stochastic games), where all agents act synchronously at each time step. In contrast, turn-based interactions, where agents take turns to act (e.g., chess, Kuhn Poker, tic-tac-toe), are typically modeled as extensive-form games.

Non-Optimality of POSG Solutions

We illustrate that the solutions of POSG, i.e., NE, may not necessarily lead to joint optimality in cooperative settings.

Consider a one-step matrix game involving 2 agents, where each agent selects an action from the action space $\mathcal{A} = \{\mathcal{A}^{(1)}, \mathcal{A}^{(2)}\}$. The joint action profile determines the utility as presented in Table 4.

$a_1 \backslash a_2$	$\mathcal{A}^{(1)}$	$\mathcal{A}^{(2)}$
$\mathcal{A}^{(1)}$	10	7
$\mathcal{A}^{(2)}$	7	0

Table 4: Joint utility matrix of 2 agents.

This matrix game can be potentially decomposed into 2 POSG in Table 5 through reward shaping.

$a_1 \backslash a_2$	$\mathcal{A}^{(1)}$	$\mathcal{A}^{(2)}$
$\mathcal{A}^{(1)}$	(5, 5)	(3, 4)
$\mathcal{A}^{(2)}$	(4, 3)	(0, 0)

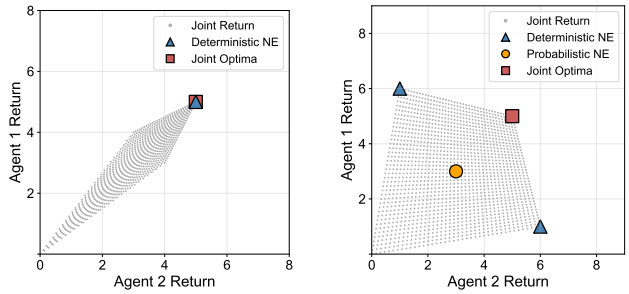
(a) POSG 1

$a_1 \backslash a_2$	$\mathcal{A}^{(1)}$	$\mathcal{A}^{(2)}$
$\mathcal{A}^{(1)}$	(5, 5)	(1, 6)
$\mathcal{A}^{(2)}$	(6, 1)	(0, 0)

(b) POSG 2

Table 5: Return tables of 2 POSG.

In the POSG presented in Table 5a, $(\mathcal{A}^{(1)}, \mathcal{A}^{(1)})$ is a Nash equilibrium (blue triangle in Figure 4a). When $a_1 = \mathcal{A}^{(1)}$, $U_2(\mathcal{A}^{(1)}, \mathcal{A}^{(1)}) > U_2(\mathcal{A}^{(1)}, \mathcal{A}^{(2)})$; when $a_1 = \mathcal{A}^{(2)}$, $U_2(\mathcal{A}^{(2)}, \mathcal{A}^{(1)}) > U_2(\mathcal{A}^{(2)}, \mathcal{A}^{(2)})$. Therefore, the best response for agent 2 is $a_2^{\circledast} = \mathcal{A}^{(1)}$. Similarly, since $U_1(\mathcal{A}^{(1)}, \mathcal{A}^{(1)}) > U_1(\mathcal{A}^{(2)}, \mathcal{A}^{(1)})$, we obtain $a_1^{\circledast} = \mathcal{A}^{(1)}$. This NE also achieves joint optimality with the maximum utility $5 + 5 = 10$ (red square in Figure 4a).



(a) POSG 1 (b) POSG 2

Figure 4: Utility spaces of 2 POSG.

However, certain reward decompositions may yield non-optimal solutions for cooperative games in Table 5, even when POSG solutions reach NE. For the POSG shown in Table 5b, the deterministic NE are $(\mathcal{A}^{(1)}, \mathcal{A}^{(2)}), (\mathcal{A}^{(2)}, \mathcal{A}^{(1)})$ (blue triangles in Figure 4b). When $a_1 = \mathcal{A}^{(1)}$, agent 2 prefers $\mathcal{A}^{(2)}$ as $U_2(\mathcal{A}^{(1)}, \mathcal{A}^{(2)}) > U_2(\mathcal{A}^{(1)}, \mathcal{A}^{(1)})$; when $a_1 = \mathcal{A}^{(2)}$, agent 2 prefers $\mathcal{A}^{(1)}$ since $U_2(\mathcal{A}^{(2)}, \mathcal{A}^{(1)}) > U_2(\mathcal{A}^{(2)}, \mathcal{A}^{(2)})$. Agent 1 faces the same issue. Thus, neither agent can unilaterally improve their utilities by deviating. However, the collective utilities obtained from both policies yield $6 + 1 = 7 < 10$, which are suboptimal compared to the joint optimum (red square in Figure 4b).

In Table 5b, even the probabilistic NE under stochastic policies is still non-optimal. Suppose agent 1 selects $\mathcal{A}^{(1)}$

Method	Dataset	Pass@k (%)			Acc@k (%)			Coop@k (%)		
		@3	@5	@10	@3	@5	@10	@3	@5	@10
Single Model	HE	67.7	71.0	83.9	85.4	87.9	**95.1**	–	–	–
	CHE	68.8	75.0	81.3	75.0	81.3	88.8	–	–	–
Naive Concatenation	HE	45.2	51.6	64.5	70.4	75.5	80.9	49.5	67.7	76.3
	CHE	43.8	56.3	68.8	57.0	63.8	73.8	47.9	69.3	81.3
Sequential Pipeline	HE	54.8	61.3	71.0	78.8	84.5	91.9	62.4	73.3	92.7
	CHE	**75.0**	**81.5**	**87.5**	**88.2**	89.5	91.3	75.0	75.0	81.3
One-Round Discussion	HE	51.6	61.3	71.0	71.8	81.3	87.5	58.1	70.0	78.7
	CHE	50.0	68.8	68.8	75.4	82.5	82.0	66.7	68.7	75.0
Single-Turn MLGRPO (Ours)	HE	54.8	58.1	71.0	75.3	76.3	86.4	83.8	90.3	90.3
	CHE	68.8	75.0	81.2	80.0	82.5	87.5	87.5	93.8	93.8
Multi-Turn MLGRPO (Ours)	HE	**71.0**	**80.6**	**90.3**	**85.7**	**92.6**	94.7	**93.5**	**96.8**	**96.8**
	CHE	**75.0**	81.3	**87.5**	86.4	**92.5**	**95.4**	**93.8**	**96.8**	**100.0**

Table 3: Performance comparison between MAGRPO and baseline methods with pass@k, acc@k, coop@k, on HE and CHE. The **bold** texts indicate the best performance of each metric on each dataset.

with probability p, and agent 2 selects $\mathcal{A}^{(1)}$ with probability q, $R_1(\mathcal{A}^{(1)}, \cdot) = 5q + (1-q) = 4q + 1$, $R_1(\mathcal{A}^{(2)}, \cdot) = 6q$, $R_1(\mathcal{A}^{(1)}, \cdot) = R_1\mathcal{A}^{(2)}, \cdot)$ yields $q = 0.5$; similarly, $R_2(\cdot, \mathcal{A}^{(1)}) = 5p + (1-p) = 4p + 1$, $R_2(\cdot, \mathcal{A}^{(2)}) = 6p$, $R_2(\mathcal{A}^{(1)}, \cdot) = R_2\mathcal{A}^{(2)}, \cdot)$ yields $p = 0.5$. This probabilistic NE, $\pi_1^{\circledast}(\mathcal{A}^{(1)}) = \pi_1^{\circledast}(\mathcal{A}^{(2)})$, $\pi_2^{\circledast}(\mathcal{A}^{(1)}) = \pi_2^{\circledast}(\mathcal{A}^{(2)})$ leads to overall utilities $3 + 3 = 6 < 10$ (orange circle in Figure 4b).

Although appropriate reward shaping techniques can transform a cooperative game into a POSG like Table 5a to make the NE also jointly optimal, this becomes more challenging when more agents are involved and episodes become longer. We employ Dec-POMDP to avoid the intricate reward engineering and seek the joint optimality.

Additional Results

We report additional results in this section to validate the effectiveness of our approach.

@k Ablation

In LLMs, single-run inference often leads to high variance. To provide a more reliable evaluation, we evaluate the pass, test accuracy, and cooperation at k runs (pass@k, acc@k, and coop@k) on MAGRPO and baselines. These @k metrics measure the best outcome among k generated solutions for each problem and are averaged over all problems in $\mathcal{D}$.

Pass@k is calculated as the probability that at least one out of k generated solutions passes all test cases (Chen et al. 2021). Specifically, a set $\mathcal{P}$ of k solutions is randomly sampled from a pool of $\mathcal{M}$ generated solutions. To make it consistent with other @k metrics, we express pass@k as,

$$\text{Pass@k} = \frac{1}{|\mathcal{D}|} \sum_{j=1}^{|\mathcal{D}|} \mathbb{E}_{\mathcal{P} \sim \text{Sample}(\mathcal{M}, k)} \left[\max_{p \in \mathcal{P}} \mathbb{1}(n_j^{\text{pass}} = n_j) \right], \tag{6}$$

where $\mathbb{1}(n_j^{\text{pass}} = n_j)$ is the indicator function that equals 1 if all test cases are passed in problem j (i.e., the number of passed tests n_j^{pass} equals the total number of tests n_j), and 0 otherwise.

However, pass@k does not offer a fine-grained assessment of solution quality, as functional correctness is represented as a binary variable. As a result, failing a single test case is treated the same as failing all in pass@k. To provide a more unbiased evaluation, we use accuracy@k, defined as the highest test accuracy among the k generated solutions, averaged across all problems.

$$\text{Acc@k} = \frac{1}{|\mathcal{D}|} \sum_{j=1}^{|\mathcal{D}|} \mathbb{E}_{\mathcal{P} \sim \text{Sample}(\mathcal{M}, k)} \left[\max_{p \in \mathcal{P}} \frac{n_j^{\text{pass}}}{n_j} \right], \tag{7}$$

where n_j^{pass} and n_j are the number of passed and total unit tests of problem j, and n_j^{pass}/n_j is the test accuracy.

Similar to acc@k, we also propose coop@k, which measures the average of the highest normalized cooperation return achieved among k runs across all problems. Formally, the Coop@k is defined as,

$$\text{Coop@k} = \frac{1}{|\mathcal{D}|} \sum_{j=1}^{|\mathcal{D}|} \mathbb{E}_{\mathcal{P} \sim \text{Sample}(\mathcal{M}, k)} \left[\max_{p \in \mathcal{P}} R_p^{\text{coop}} \right], \tag{8}$$

where $R_p^{\text{coop}} = \sum_{t=0}^{H-1} r_{p,t}^{\text{coop}}$ is the cooperation return over horizon H, and $r_{p,t}^{\text{coop}}$ denotes the cooperation reward obtained by solution p at turn t.

We generate 15 samples in $\mathcal{M}$ and evaluate all methods with $k = 10$. Table 3 presents the results for pass@k, acc@k, and coop@k at $k = 3, 5, 10$, comparing MLGRPO with baseline methods. As expected, the trends in @k metrics are consistent with the @1 results. The naive concatenation method remains worse than the single-agent baseline in terms of pass@k, as the main agent may generate code based on incorrect assumptions about the auxiliary function. The sequential pipeline mitigates this issue by allowing the main agent to reference the auxiliary output during generation, yielding substantial improvements across all @k metrics, particularly on CHE. Although the one-round discussion method shares the same prompts as naive concatenation in the first turn, the additional discussion round yields limited improvement across the @k metrics due to misalignment in cross-adaptation.

Multi-turn MAGRPO outperforms all baselines across most @k metrics on test and cooperation by incorporating rational feedback from the external model. These more comprehensive experiments further demonstrate the general effectiveness of our approach and motivate future efforts to train LLMs under expert guidance. Notably, the acc@k provides a more fine-grained view of performance trends with respect to k. In some cases, subtly increasing k may not lead to more solutions passing all test cases (as measured by pass@k), but the improvement is reflected in higher accuracy.

Cooperation Schemes

By training the auxiliary and main coders to cooperate under minimal constraints (with only the problem description and their respective roles provided), diverse cooperation schemes naturally emerge. We present 4 representative schemes observed in our models.

Fallback The most commonly observed cooperation scheme is the main agent providing a fallback for the auxiliary function. Although prompted to use the auxiliary output, the main agent may still doubt its completeness and reimplement the solution as a backup in case of potential errors.

For example, the problem `compare_one(a,b)` asks to return the larger of 2 values in its original type (string, float, or int), or `None` if equal. Listing 1 demonstrates how the fallback mechanism provides an alternative in case of runtime errors in the auxiliary implementation. The `aux(x,y)` function converts variable `x` to a float, resulting in an operand type mismatch if `y` is a string. The main agent handles the type error by falling back to its secondary implementation. While the main implementation may still be vulnerable, this mechanism benefits small models that commonly encounter syntax and runtime errors. Despite some redundancy, a proper fallback can enhance code robustness.

Decorator When the main agent trusts the auxiliary to accurately handle the core logic, it typically acts as a decorator, adding complementary features to enhance the output. The main agent could handle edge cases for improved robustness or reformat the outputs (e.g., sorting, rounding, and adjusting units). The necessity of such decorating logic depends on the auxiliary agent's functionality and completeness.

Listing 2 illustrates a decorator main function and how its utility varies depending on the completeness of the auxiliary implementation. In this problem, 2 agents generate codes together to check if a given string is a palindrome. When paired with `aux1` that only handles the recursion boundary condition of single-character strings, the empty string check of the main serves as a necessary edge case handler. However, when working with `aux2`, which already has a more comprehensive edge case consideration, this handle becomes redundant.

Listing 1: `compare_one(a,b)`

```
def aux(x, y):
    # Convert x to a float
    if isinstance(x, str):
        x = float(x.replace(",", "."))

    # Vulnerable if y in different types
    if x < y:
        return y
    else:
        return x

def compare_one(a, b):
    try:
      return aux(a, b)
  except Exception:
      # Robust implementation
      if type(a) == type(b):
          if a > b:
                return a
          elif a < b:
                return b
          else:
                return None
      elif type(a) == float and type(b) == str:
          b_float = float(b.replace(",", "."))
          if a > b_float:
                return a
          elif a < b_float:
                return b_float
          else:
                return None
      elif type(a) == str and type(b) == float:
            a_str = str(a.replace(",", "."))
            a_float = float(a_str)
          if a_float > b:
                return a_str
          elif a_float < b:
                return b
          else:
                return None
      else:
          return None
```

Coordinator In large-scale software systems, it would be beneficial to have pipelines for repeated or data-parallel operations (e.g., batch processing, stream transformations). This corresponds to the coordinator cooperation scheme in our models, where the main agent divides the tasks into parts and assigns them to the auxiliary agent.

A simple example involves the main agent acting as an iterator, using a loop (e.g., a `for` loop) to structure the problem. The auxiliary function generates partial solutions within each iteration. These partial results are then aggregated to form the final output. However, this cooperation scheme is unstable, as it depends heavily on the correct functionality of the auxiliary agent. When the auxiliary agent fails to complete its subtask, the entire solution breaks down.

Listing 2: `is_palindrome(text)`

```
def aux1(text):
    if len(text) == 1:
        return True
    else:
        return text[0] == text[-1] and aux(text[1:-1])

def aux2(text):
    if len(text) <= 1:
        return True
    else:
        return text[0] == text[-1] and aux(text[1:-1])

def is_palindrome(text):
    if not text:
        return True

    # Edge case handler
    return text == aux1(text)

    # Redundant decorator
    return text == aux2(text)
```

Listing 3: `flip_case(string)`

```
def aux(string: str) -> str:
   result = ""
   for char in string:
       if char.islower():
           result += char.upper()
       elif char.isupper():
           result += char.lower()
       else:
           result += char
   return result

def flip_case(string: str):
   flipped = ""
   for char in string:
       flipped += aux(char)
   return flipped
```

Listing 3 demonstrates a solution to flip the case of characters in a string. The auxiliary function flips the case of each character, while the main function calls this auxiliary function for each character and appends it to the result. This scheme can be extended to more complex scenarios, where subtasks are assigned in a hierarchical structure.

Strategy Filter When handling complex problems, the main agent may need to implement logic based on multiple conditions. In such cases, the auxiliary agent can act as a filter for specific branches of logic, often appearing within conditional blocks (e.g., following an `if` statement). This scheme resembles the adaptive control flow in practice. In rule-based pipelines, an auxiliary agent evaluates preconditions (e.g., task types, system status, configurations) and directs workers to execute appropriate subroutines, thereby enhancing project modularity.

Listing 4: `x_or_y(n,x,y)`

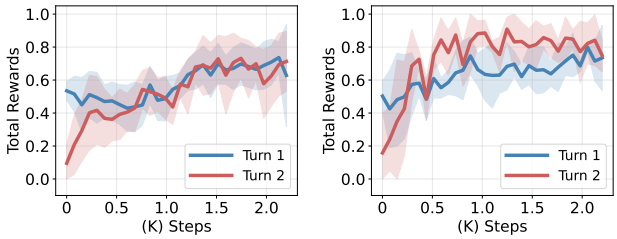
```
def aux(n):
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True

def x_or_y(n, x, y):
    # Check if n is prime
    if aux(n):
        return x
    else:
        return y
```

Listing 4 presents a solution for `x_or_y(n,x,y)` problem, which returns `x` if `n` is prime and `y` otherwise. The auxiliary function handles the primality checking, while the main function is responsible for returning results. The same pattern can also be found in the solutions of `prime_fib(n)`, `factorize(n)`, and `largest_prime_factor(n)`.

Learning Modes

In the multi-turn MAGRPO training, agents learn to cooperate through different learning modes. Figure 5 shows the curves of total returns in a 2-turn MAGRPO training.



(a) Self Learning (b) Guided Learning

Figure 5: Learning modes in 2-turn MAGRPO training with external feedback from `Claude-Sonnet-4`.

Figure 5a demonstrates a self-learning mode, where agents primarily interact with the tasks themselves and absorb little from external feedback. At the beginning, the second-turn rewards (red) are lower than the first-turn rewards (blue), as agents struggle to incorporate feedback effectively. With training, both curves improve as agents gradually develop cooperative behaviors. Nevertheless, the performance of the second turn is still consistently lower than the first turn, suggesting that learning is primarily driven by direct task interaction rather than external guidance. This pattern typically arises when external feedback is ineffective or poorly interpreted by the agents.

Figure 5b illustrates guided learning, where LLMs leverage external feedback to improve performance. When using *Claude-Sonnet-4* as an external model to provide more concrete suggestions (e.g., code edits), the performance of the second turn (red) exceeds first turn (blue), and both outperform those in the self learning. This indicates that appropriate guidance helps agents to refine the response in an efficient way. Due to the computational constraints, most models used in our setup are around 3B parameters and may struggle to interpret more abstract suggestions. We hypothesize that larger models with higher reasoning capabilities could benefit from more implicit guidance.

Experimental Configurations

This section outlines the hyperparameter settings and reward specifications used in our experiments.

Hyperparameters

For writing collaboration, we set the temperature to 0.8 and apply nucleus sampling with a threshold of 0.95 to encourage diverse generation. Policy deviation is regularized using a beta value of 0.02. The policy is optimized using a learning rate of 5×10^{-6}, and training is conducted for 1,500 steps.

For coding collaboration, the single-turn MAGRPO training uses a temperature of 0.7 and nucleus sampling with a threshold of 0.9. The learning rate is set to 1×10^{-6}, with 1,500 training steps. In the multi-turn setting, the discount factor is set to 1.0, and the learning rate is 5×10^{-6}, with 2,200 training steps.

Reward Specifications

Rewards are computed as a weighted sum of multiple metric-based components, following a hierarchical reward modeling scheme to prioritize fundamental objectives.

TLDR Summarization

- **Structural Wellness**: The structural wellness is assessed by the ratios of paragraph length and unique words. For the completion length, an appropriate ratio within 1.6-3.2× receives the full rewards; ratios within the range of 1.1-5.0× receive proportional rewards; while ratios outside receive no rewards and early termination of evaluation. For the ratio of unique words, we exclude the common stopwords. A ratio of 2.0× or higher receives the maximum rewards; ratios between 1.3-2.0× receive proportional rewards; ratios below 2.0× result in no rewards and evaluation termination.
- **Style Consistency**: The style consistency is measured through Jaccard similarity of vocabulary between the completions (excluding stopwords). The Jaccard similarity scores are capped at 0.03 and normalized as rewards. We use a cap here to balance the needs of maintaining lexical consistency and vocabulary expansion in elaborated summarization.
- **Logical Coherence**: The logical coherence is evaluated through the presence and diversity of transition words in the completions. We check transition words across 12 categories, e.g., examples, explanation, contrast, etc. Additional rewards are given for using transition words in more categories, where $r = \min(0.6 \log(n+1), 1)$, and n is the number of transition categories.

arXiv Expansion

- **Structural Wellness**: This metric evaluates the relative length and lexical diversity between the second and first completions. A length ratio within the optimal range of 1.0–1.3× yields the maximum rewards, while ratios within the acceptable bounds of 0.8–1.5× receive proportionally scaled rewards. Ratios outside this range result in zero reward and early termination. Similarly, a unique word ratio within 0.7–1.3× receives the full rewards, ratios within 0.5–1.7× are rewarded proportionally, and values outside this range lead to zero reward and evaluation termination.
- **Style Consistency**: Style consistency is quantified using Jaccard similarity between the 2 completions. The similarity score is capped at 0.23 and normalized as rewards.
- **Logical Coherence**: Logical coherence is assessed based on the presence of transition words across 12 categories. Additional rewards are given for using transition words in more categories, where $r = \min(0.4 \log(n+1), 1)$, and n is the number of transition categories.

Coding Collaboration

- **Structural Integrity**: This metric verifies the correct implementation of both the auxiliary and main functions. To receive the base reward, the corresponding functions in the agents' completions must be properly defined and include return statements. Failure to define the main function results in evaluation termination.
- **Syntactical Correctness**: This metric assesses the syntactic validity of the concatenated code, which includes the libraries provided in the dataset, the auxiliary function defined by the helper agent, and the function defined by the main agent. Syntactic correctness is verified via static analysis, i.e., Abstract Syntax Tree (AST). The presence of syntax errors leads to the termination of the evaluation to avoid runtime failures.
- **Test Pass Rate**: This metric measures the percentage of unit tests passed during execution, with each test subject to a 10-second timeout. Rewards are assigned proportionally based on the number of successful assertions. If no tests pass, the evaluation is terminated.
- **Cooperation Quality**: A base bonus is applied if the main function calls the auxiliary. Additional rewards are given when the main function implements substantive logic beyond simply wrapping the auxiliary.

Prompt Design

Writing Collaboration

TLDR In the TLDR summarization, the `prompt` field of the dataset is processed for each agent by using the following instructions.

Summary Agent

```
Create a concise summary response to
    this post.
Query: {prompt}
Instructions: Provide a brief and
    focused summary in a few sentences
```

Elaboration Agent

```
Create a detailed summary response to
    this post.
Query: {prompt}
Instructions: You should use transition
    words to improve flow
```

arXiv In the arXiv paragraph expansion, we use the `abstract` field of the dataset and process it as follows.

Background Agent

```
Based on the following scientific
    abstract, expand the content for the
    introduction section.
Abstract: {abstract}
Instructions:
- There is another agent that will
    provide the method and implications
- You just need to focus on the
    background and motivation
- Avoid repeating methodology and
    implications content
```

Method Agent

```
Based on the following scientific
    abstract, expand the content for the
    introduction section.
Abstract: {abstract}
Instructions:
- There is another agent that will
    provide the background and motivation
- You just need to focus on the method
    and implications
- Avoid repeating background and
    motivation content
```

Coding Collaboration

For HE and CHE, we extract the `entry_point`, `params` from the `prompt` field and instruct agents as follows.

Auxiliary Agent

```
Create a helper function for this coding
     problem.
Problem: {prompt}
Instructions:
- Output ONLY the function code, no
    explanations or examples
- Do NOT include markdown code blocks
    (```python)
- Do NOT include any text before or
    after the function
- Do NOT include test cases or example
    usage
- Create a helper function named 'aux'
    that can assist the main function
- The function should return useful data
     for solving the problem

Your output should follow this format:
def aux(...):
    # your code here
    return result
```

Main Agent

```
Solve this coding problem by
    implementing the required function.
Problem: {prompt}
You have access to a helper function:
    aux(...)

Instructions:
- Output ONLY the function code, no
    explanations or examples
- Do NOT include markdown code blocks
    (```python)
- Do NOT include any text before or
    after the function
- Do NOT include test cases or example
    usage
- Do NOT redefine the aux() function
- Implement ONLY the '{entry_point}'
    function as specified
- You can call aux() to assign a value
    to a variable within your function if
     helpful

Your output should follow this format:
def {entry_point}({params}):\n # your
    function code here\nreturn result\n
```

To improve the generated code, these prompts are used to construct second-turn observations for the MAS with suggestions from *Claude-Sonnet-4*.

External Agent

```
You are an advisor helping 2 agents (an
    auxiliary agent and a main agent)
    solve the following problem. The
    auxiliary agent provides a helper
    function (aux), while the main agent
    defines the task-specific logic.
Problem: {prompt}
Example tests: {test}
Show your feedback and edits for the
    following code: {combined_code}

Instructions:
- If you identify a missing element,
    such as an undefined aux or missing
    entry point (main function), you
    should write one for it.
- If both are not missing, point out and
     make changes to any critical syntax
    or logic errors that would prevent
    the code from passing the given unit
    tests.
- You should focus only on clear errors
    on the given unit tests, be
    conservative and lenient, ignoring
    issues like redundancy, inefficiency,
```

```
     lack of edge case handling, or type
    annotations.
- Return "Perfect! No edits needed!" if
    the logic is sound.

Your response MUST contain the JSON
    format specified below. Always
    include both 'aux' and 'main' fields,
     even if no edits are needed.
{ "aux": {{...}}, "main": {{...}}}
```

Baseline Methods

We adopt a single-agent method and 3 representative multi-agent conversation methods as baselines. We provide details of these baseline approaches below.

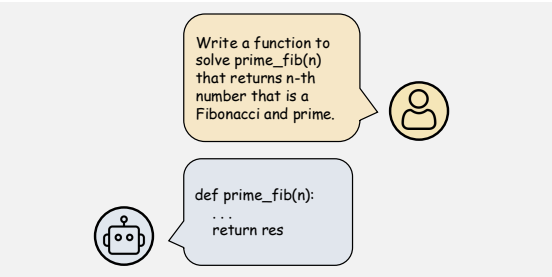


Figure 6: Single-agent code generation.

Figure 6 illustrates the code generation process using a single LLM agent. In this setting, the user gives a coding question along with specific instructions. The agent responds by generating a Python function snippet to solve it.

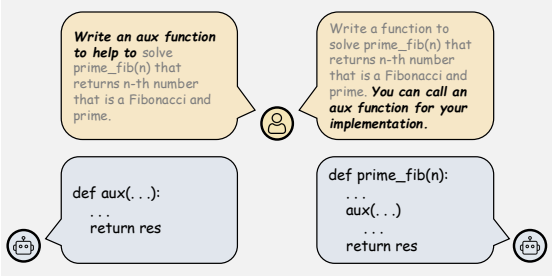


Figure 7: Coding collaboration via naive concatenation.

Naive concatenation represents the simplest form of cooperation, where 2 agents generate code synchronously, as illustrated in Figure 7. The first agent is provided with the coding question and informed of its role as a helper. The second agent is given the same question, along with its role as the main generator and the fact that an auxiliary agent exists. However, the main agent lacks information about the specific functionality of the auxiliary agent. Their outputs are directly concatenated to form the response. This method is intended to improve inference efficiency by enabling a simple division of the problem into separate parts.

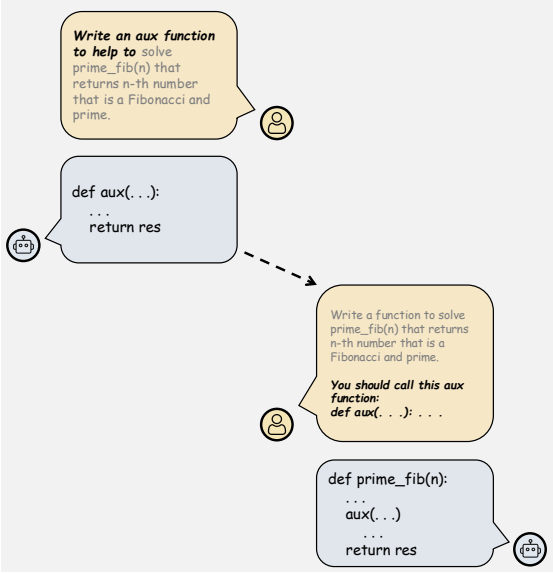


Figure 8: Coding collaboration via sequential pipeline.

Figure 8 presents the form of pipeline cooperation, where agents respond in sequence. In this setting, the first agent is given the coding question along with the role of a helper. Its response is then passed to the main agent as a reference. The main agent generates its answer by incorporating the helper's response. This method enables one-way communication, allowing the main agent to respond by coordinating with the helper. However, this comes at the cost of slower inference speed due to the sequential nature of the interaction.

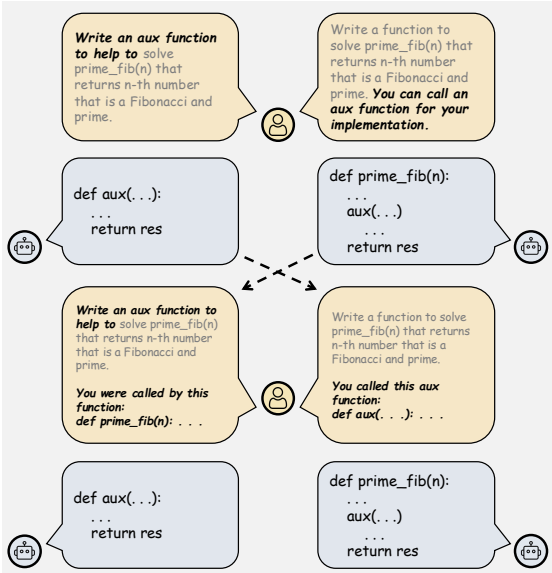


Figure 9: Coding collaboration with one-round discussion.

Discussion or debate frameworks (Figure 9) aim to improve response quality by enabling agents to access each

other's previous outputs (Du et al. 2023; Liang et al. 2024). In the first turn, the helper and main agents generate responses in the same manner as the naive concatenation approach. These initial responses are then shared with each other as references for the following turns, forming a discussion. Although this setup introduces more interaction, it does not guarantee improved response quality. With a limited number of rounds, the final output may not converge to a coherent solution. Even with additional rounds, convergence remains uncertain. This approach can even be less efficient than the sequential pipeline, particularly in distributed systems where communication latency is high or unreliable.

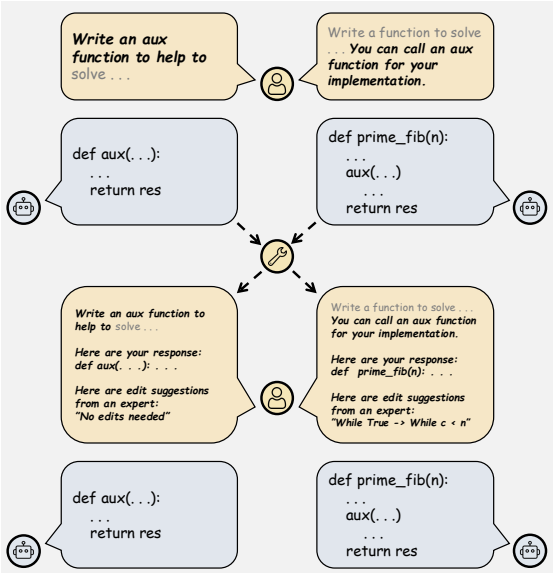


Figure 10: Coding collaboration in our method.

The interaction process between 2 agents trained with MAGRPO is illustrated in Figure 10. In the single-turn setting, we use the same prompts as in the naive concatenation baseline. In the multi-turn setting, after the helper and main agents generate their initial responses, these outputs are reviewed by an external agent. In this work, we employ *Claude-Sonnet-4* as an external to provide edit suggestions. For each agent, the suggestions, as well as the prior information and their previous response, are incorporated into the prompt for the subsequent round.

Note that the baselines above can also be fine-tuned by MARL. However, the interactions among agents in these settings are not strictly cooperative, which may lead to instability during training. To address this, techniques such as role-based rewards (Liu et al. 2025), partial MAS training (Estornell et al. 2025), and freezing selected agents (Subramaniam et al. 2025) can be employed to ensure stable optimization.

Broader Impacts

Prompt-based coordination is often brittle (Estornell and Liu 2024), as agents may fail to follow instructions they were not explicitly trained to interpret. Our method builds on a solid theoretical foundation in cooperative MARL, explicitly optimizing agents for joint optimality. Our work also opens opportunities to enhance existing test-time multi-agent interaction methods by integrating MARL techniques (Du et al. 2023; Lifshitz, McIlraith, and Du 2025; Wu et al. 2023a), particularly in settings that involve task decomposition and iterative feedback integration.

This work also explores a new perspective on accelerating LLM inference through cooperative MARL. While mainstream acceleration techniques (e.g., knowledge distillation, pruning, and quantization) improve efficiency at the cost of information loss (Wang et al. 2024; Zhao et al. 2024), our approach suggests decentralized coordination among specialized agents, thereby alleviating the burden of long-context memory and joint decision-making on a single model. Each agent can focus on a specific subtask, enabling more modular and robust reasoning.

Limitations and Future Works

Nevertheless, this study is subject to several limitations. First, we focus on homogeneous agents for simplicity, assuming they perform similar tasks despite being assigned different roles, e.g., both the auxiliary agent and main agent are generating Python functions. Future research could explore LLM collaboration among heterogeneous agents with diverse capabilities and functionalities.

Due to computational constraints, we train LLMs with MAGRPO on limited datasets using relatively small-scale language models. When LLM-based coding agents are deployed in larger-scale projects involving multiple files and modules, more diverse and complex cooperation schemes are likely to emerge, which would further demonstrate the potential of decentralized coordination in MAS.

The simplicity of our reward model inevitably leads to narrow reward signals and potential reward hacking. As suggested by many research studies and industrial practice (Uesato et al. 2022; Wu et al. 2023b; Anthropic 2023), designing more expressive and fine-grained reward models (e.g., multi-aspect rewards, process-supervised rewards) is essential for better aligning agent cooperation with human preferences.

Compute Resources

We use H200 GPUs for LLM training, and a standalone NVIDIA GeForce RTX 5090 workstation for the inference of our models and the baseline methods. Here are the specifications of the resources we used in our experiments.

```
Training Devices
Type: GPU Cluster
CPU:  Intel Xeon Platinum 8558
GPU:  1x NVIDIA H200

Inference Device
Type: Standalone Workstation
CPU:  AMD Ryzen 9 9950X (16-Core) 5.7
    GHz Turbo (Zen 5)
GPU:  1x NVIDIA GeForce RTX 5090
```